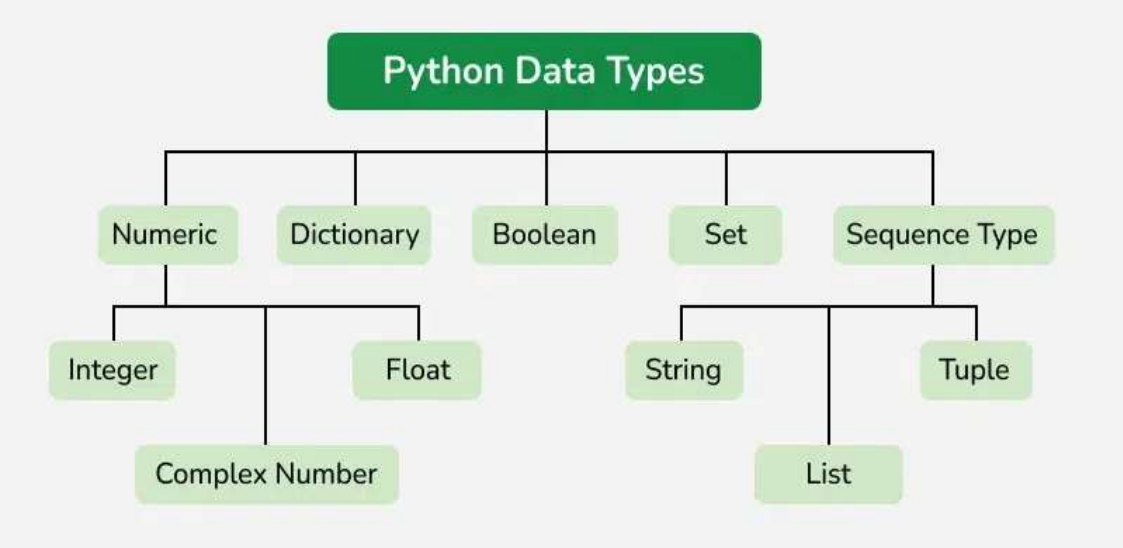
Day-1 : Python

Introduction:

Python is one of the most popular programming languages. It's simple to use, packed with features and supported by a wide range of libraries and frameworks. Its clean syntax makes it beginner-friendly.

- A high-level language, used in data science, automation, AI, web development and more.
- Known for its readability, which means code is easier to write, understand and maintain.
- Backed by strong library support, we don't have to build everything from scratch.

Python Data Types:



Conditional Statements in Python:

Conditional statements are used to control the flow of execution in a program based on specific conditions. They allow programs to execute different blocks of code depending on whether a condition evaluates to True or False.

(i)If Statement:

If statement is used to execute a block of code only when a specified condition evaluates to True.

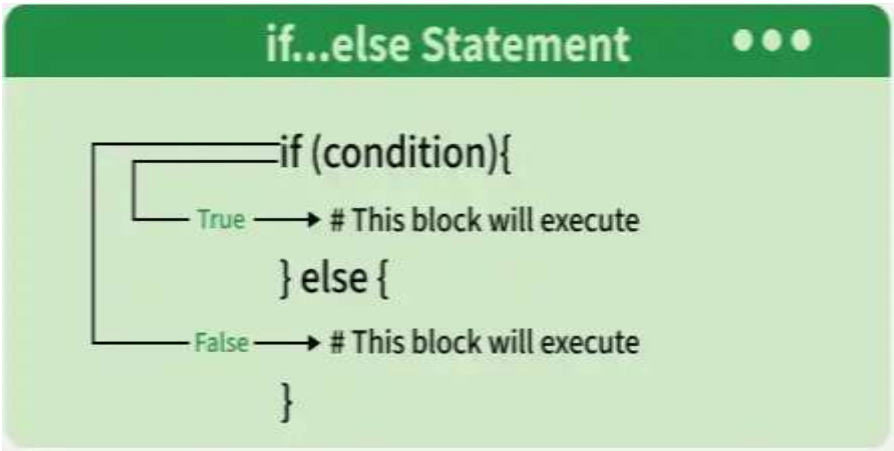


Example:

```
age = 20
if age >= 18:
    print("Eligible to vote.")
```

(ii)If else Statement:

If Else statement is used to execute one block of code when the condition is True and another block when the condition is False.

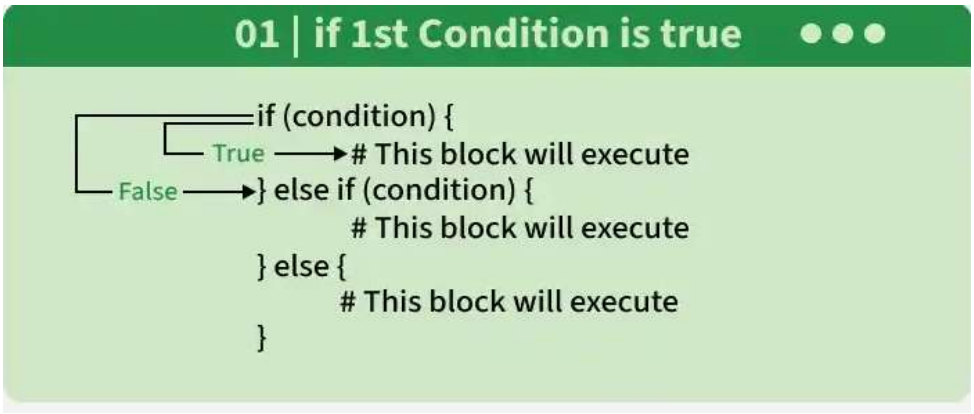


Example:

```
age = 10
if age <= 12:
    print("Travel for free.")
else:
    print("Pay for ticket.")
```

(iii)If-elif-else Statement:

elif statement is used to check multiple conditions in a program. It executes a block of code when its condition evaluates to True after previous conditions evaluate to False.



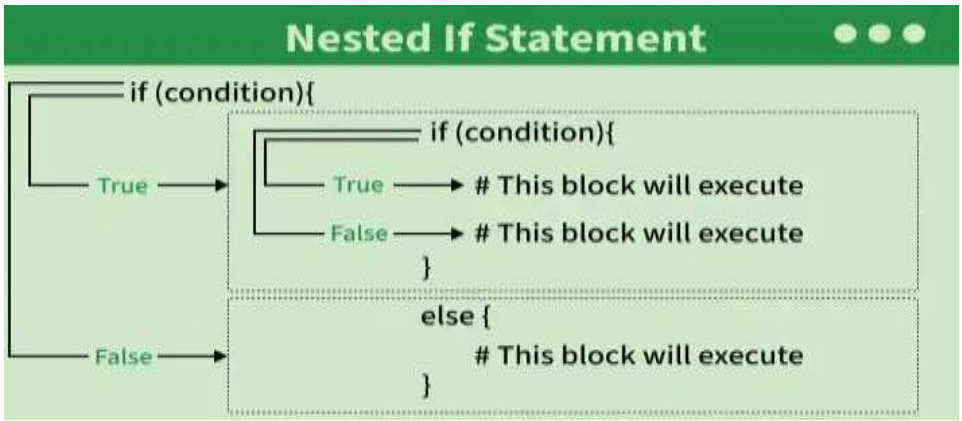
Example:

```
age = 25

if age <= 12:
    print("Child.")
elif age <= 19:
    print("Teenager.")
elif age <= 35:
    print("Young adult.")
else:
    print("Adult.")
```

(iv)Nested if-else Statement:

Nested if-else statement is an if-else statement placed inside another if or else block. It is used to check conditions within another condition.



Example:

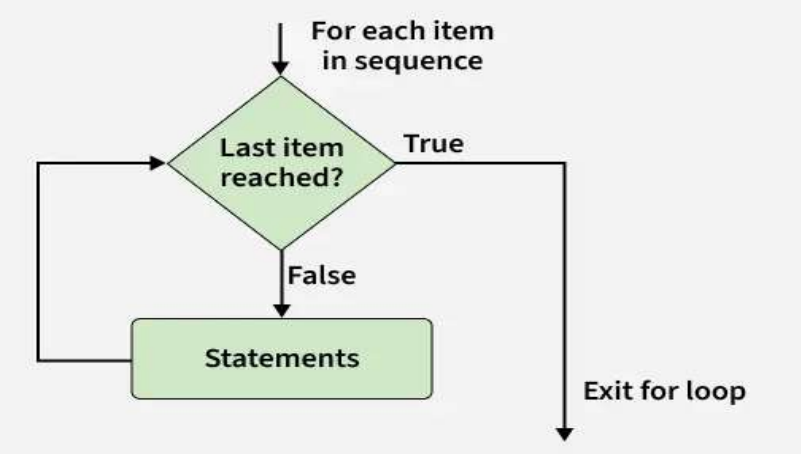
```
age = 70
is_member = True
if age >= 60:
    if is_member:
        print("30% senior discount!")
    else:
        print("20% senior discount.")
else:
    print("Not eligible for a senior discount.")
```

Loops in Python:

Loops are used to execute a block of code repeatedly until a condition is met or all items in a sequence are processed. The main types are For loops (iterating over sequences) and While loops (executing code based on a condition).

(i)For Loop:

For loops is used to iterate over a sequence such as a list, tuple, string or range. It executes a block of code once for each item in the sequence

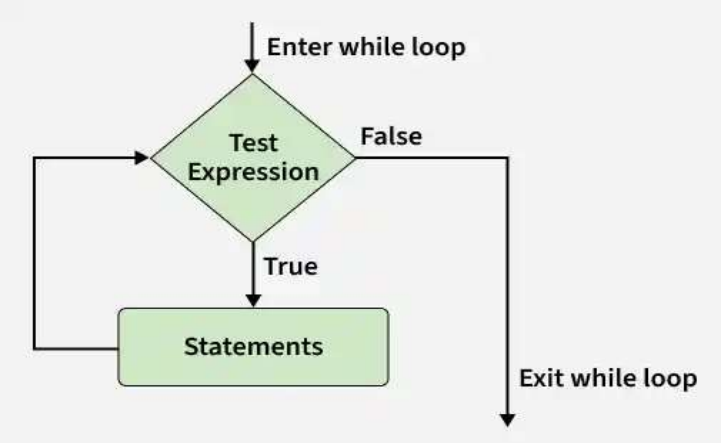


Example:

```
n = 4
for i in range(0, n):
    print(i)
```

(ii) While Loop:

While loop repeatedly executes a block of code as long as the given condition remains true. When the condition becomes false, the line immediately after the loop in the program is executed.



Example:

```
cnt = 0

while (cnt < 3):

    cnt = cnt + 1

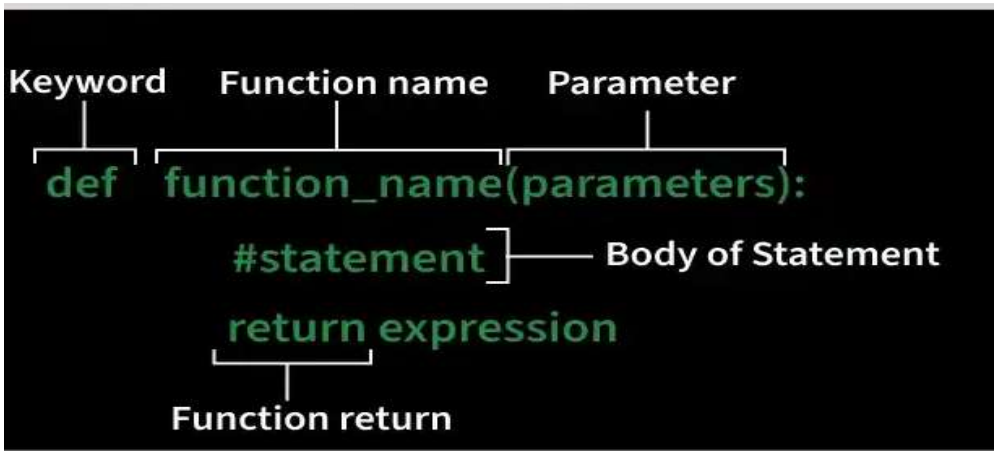
    print("Hello Geek")
```

Python Functions:

Python functions are reusable blocks of code used to perform a specific task. They help organize programs into smaller sections and execute the same logic whenever needed by calling the function.

Defining a Function

A function can be defined using def keyword. Below is the syntax to define a function:



Example:

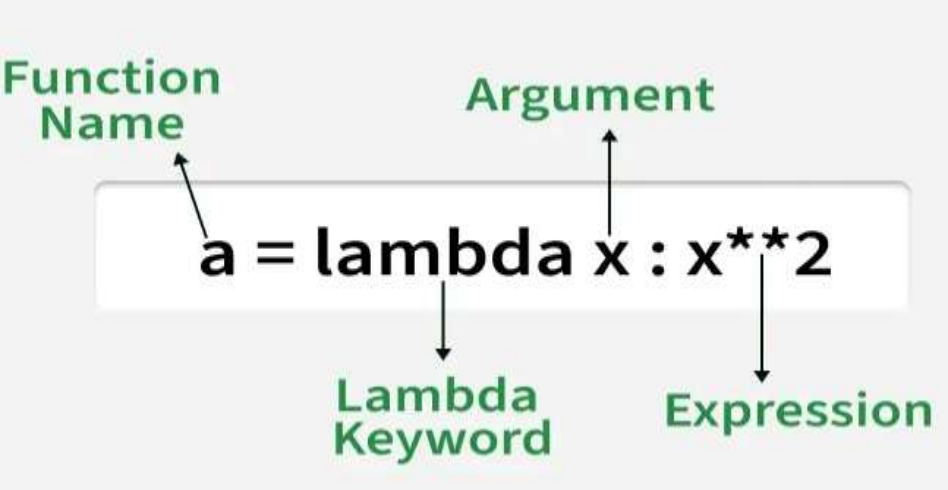
```
def evenOdd(x):
    if (x % 2 == 0):
        return "Even"
    else:
        return "Odd"


print(evenOdd(16))
print(evenOdd(7))
```

Python Lambda Functions:

Lambda functions are small anonymous functions, meaning they do not have a defined name. These are small, short-lived functions used to pass simple logic to another function.

- Contain only one expression.
- Result of that expression is returned automatically (no return keyword needed).



Example:

In this example, a lambda function is defined to convert a string to its upper case using upper().

```
a = 'JyotiVerma'

upper = lambda x: x.upper()

print(upper(a))
```

Recursion in Python:

Recursion is a programming technique where a function calls itself either directly or indirectly to solve a problem. It is commonly used for:

- Breaking problems into smaller subproblems
- Mathematical calculations like factorial and Fibonacci
- Tree and graph traversal and Divide-and-conquer algorithms
- A recursive function calls itself in its body. Let's see basic structure of recursive function:
- def recursive_function(parameters):
  if base_case_condition:
  return base_result
  else:
  return recursive_function(modified_parameters)
- Recursive function contains two key parts:
  - **Base Case:** stopping condition that prevents infinite recursion.
  - **Recursive Case:** part of the function where it calls itself with modified parameters.

Example:

1)

```
def factorial(n):
    if n == 0:  # Base case
        return 1
    else:       # Recursive case
        return n * factorial(n - 1)
print(factorial(5))
```

2)

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
print(fibonacci(10))
```